

Task: AI-powered Smart Inquiry Triage Assistant

Topic	Details
Position	Intern AI Engineer – Customer Journey Analytics & Data Science
Deliverables	Part A: GitHub Repository (Code) · Part B: Presentation (max. 20 minutes)
Deadline	All commits pushed to a public GitHub repository before the agreed deadline

Context & Scenario

You are joining the **Customer Journey Analytics & Data Science** team at a large automotive company. The team is responsible for data-driven analysis and optimization of digital customer channels – including websites, apps, vehicle configurators, and dealer systems.

Every day, the company receives **hundreds of customer inquiries** across these channels – questions about vehicle features, service appointments, warranty terms, ordering, and technical issues. Today, these inquiries are sorted and routed manually: someone reads each message, decides what it's about, how urgent it is, and who should handle it. This is slow, inconsistent, and doesn't scale.

Your task: Build a prototype of an **AI-powered Smart Inquiry Triage Assistant** that takes a customer inquiry and **automatically classifies it, assigns a priority, and provides short resolution notes** – using similar past cases as context.

Important: The focus is on **technical implementation, architecture, and your decision-making process**, not on perfect answer quality. Lightweight models are perfectly fine, whether local or via an API. We want to see that you understand the concepts and can explain **why** you chose what you chose.

Part A: Technical Implementation

A1 – Setup & Knowledge Base

1. **Set up an LLM** — either a local model via **Ollama** or any LLM API of your choice:
 - A **Chat model** — a lightweight model works
 - An **Embedding model**
2. Use the provided data:
 - `data/past_cases.csv` — historical customer inquiries with `category` , `priority` , and `routed_queue`
 - `data/taxonomy.json` — the canonical list of categories, each with a `name` , `description` , and `keywords`
3. Implement a **Data Ingestion Pipeline**:
 - Load and process the past cases
 - Generate embeddings using your chosen embedding model
 - Store embeddings in a **Vector Store** — we recommend **ChromaDB** for simplicity, but you may use alternatives

A2 – Triage Pipeline (LangGraph)

Build the core triage logic as a **LangGraph** workflow:

1. **Classify**: Determine the **category** of the inquiry.
2. **Retrieve Top-K**: Use similarity search over `past_cases.csv` to find the most relevant **past cases** for the inquiry.
3. **Determine priority**: Derive the **priority** (low / medium / high) for the inquiry based on the retrieved Top-K similar cases.
4. **Route**: Direct the inquiry to the correct **queue / team** based on its category.
5. **Resolution notes**: Generate **short resolution notes (1–2 lines)** using the chat model's general knowledge, informed by the category, priority, and retrieved past cases as context.

Requirements:

- Number of retrieved past cases (Top-K) should be **configurable**.
- A **confidence threshold** should be **configurable** and used to decide when to escalate an inquiry to human review (escalate when your internal confidence signal falls below the threshold).
- The classification and routing should produce **structured output** (e.g. category, priority, target queue).

Confidence & escalation — your call: How you compute the **confidence score** is **left to you**. You may derive it from retrieval similarity, Top-K category agreement, an LLM self-report, or a hybrid — whatever you prefer. It is shown in the output and also drives the escalation decision against the configurable `confidence_threshold`. You must **justify your choice and trade-offs** in the README.

Output format: Regardless of your internal implementation, the final output for a given inquiry must always be presented in this fixed format:

```
query: <the original customer inquiry>
category: <predicted category>
priority: <low | medium | high>
routed queue: <target queue/team>
confidence: <0.0 – 1.0 confidence score>
resolution notes: <1-2 line suggested resolution>
retrieved past cases: <list of the Top-K similar past inquiries used as context>
```

A3 – Triage Interface

A sample ready-made **Streamlit** chat interface is already provided in `app/app.py`. It includes:

- Input field to paste a customer inquiry
- Display of the result in the fixed **query / category / priority / routed queue / confidence / resolution notes / retrieved past cases** format described above
- Chat / session view so multiple inquiries can be triaged in one session

Your task is to wire the interface to your triage pipeline by implementing the `triage_inquiry(query, top_k, confidence_threshold)` backend function (you may call your implementation from `src/main.py`). You do not need to build the UI from scratch.

A4 – Documentation

Make sure to properly document your code: a README with setup steps, how to run the project, the models you used, and a short explanation of your key design decisions (e.g. how you classify, how you derive priority).

Part B: Presentation

Create a **presentation (max. 20 minutes)** for a **mixed audience** — both business stakeholders and technical team members should be able to follow along.

The goal is to show that you understand not just **how** to build this, but **why** it matters and where it could go from here.

What we'd like to see:

- **Business perspective:** What problem does automated inquiry triage solve? Why is this relevant for an automotive company? What value does it create for response time, consistency, and customer experience?
- **Solution & Architecture:** High-level overview of your triage architecture. Why this design? Why a graph-based workflow? Explain your key technical decisions and trade-offs.
- **Demo & Results:** Show your prototype in action. What works well, what are the limitations?
- **Roadmap:** Where could this go next?
- **Summary & Q&A**

How to Submit

1. Create your own **public** GitHub repository
2. Work in your own repo — commit and push your code as you go (Make sure to also push the presentation!)
3. When you're done, share the **link to your repository** with us
4. **All commits must be pushed before the deadline** — late submissions will not be considered

Tips

- **Prioritize a working end-to-end pipeline** over perfection in any single component. A rough-but-complete "classify → retrieve → determine priority → resolution notes" flow beats one perfect step.
- **Use small models** (1B–4B params). Reply quality is secondary. We evaluate your implementation and understanding.
- **Explain your decisions** — we want to understand your thought process.

Good luck!